

Storage Layouts for PageIndex Subtree Retrieval

Junyao Dong

Abstract. PageIndex subtree retrieval reads Structure and Metadata but not leaf Text. This report asks whether Structure and Metadata should be colocated and, after selecting the faster layout, why MongoDB remains slower than PostgreSQL. We evaluate two layouts on the same ten-million-node tree and 200 paths in MongoDB, PostgreSQL, and SQLite. Colocation lowers P50 and P95 latency in all three engines. In an interleaved causal run, mean latency is 77.8ms in MongoDB and 59.7ms in PostgreSQL. Matched zero-fetch, ID-only, cursor-batch, and scalar-count interventions show that the gap persists without document access, returned rows, or an extra `getMore`. Direct count and key-shape controls locate the difference in per-key index processing. PostgreSQL performs the analogous B-tree, index-only-scan, and aggregate steps; it does not skip them. Profiling the scalar control locates MongoDB’s excess CPU in per-key WiredTiger cursor/key extraction plus classic `COUNT_SCAN` bookkeeping. Client record processing adds a separate 5.3ms penalty.

1 Introduction

PageIndex stores three kinds of information: tree Structure, descriptive Metadata, and leaf Text. A subtree request needs Structure to identify and order descendants and Metadata to return their titles and summaries. It does not need Text. This access pattern creates a physical-design choice: colocate Structure and Metadata for one range scan, or separate them and retrieve Metadata after the Structure scan.

We answer two questions. First, which layout gives lower subtree latency across MongoDB, PostgreSQL, and SQLite? Second, under the better layout, what causes the remaining MongoDB–PostgreSQL latency gap? The first question is an end-to-end layout comparison. The second requires causal interventions because a broad “database fetch” timer mixes index execution, document access, transfer, decoding, and record materialization.

The benchmark is deliberately narrower than the full storage API. Its range is root-exclusive and unbounded in depth, whereas the current `get_subtree` API includes the root and applies a depth limit. The results therefore characterize descendant range retrieval, not complete API latency.

2 Layouts and query paths

Both layouts keep leaf text out of the subtree scan. They differ only in whether the fields returned by `get_subtree` are kept with the tree structure. MongoDB implements each physical group as a collection. PostgreSQL and SQLite implement each group as a table. Text remains separate in both layouts and is read only when content is requested.

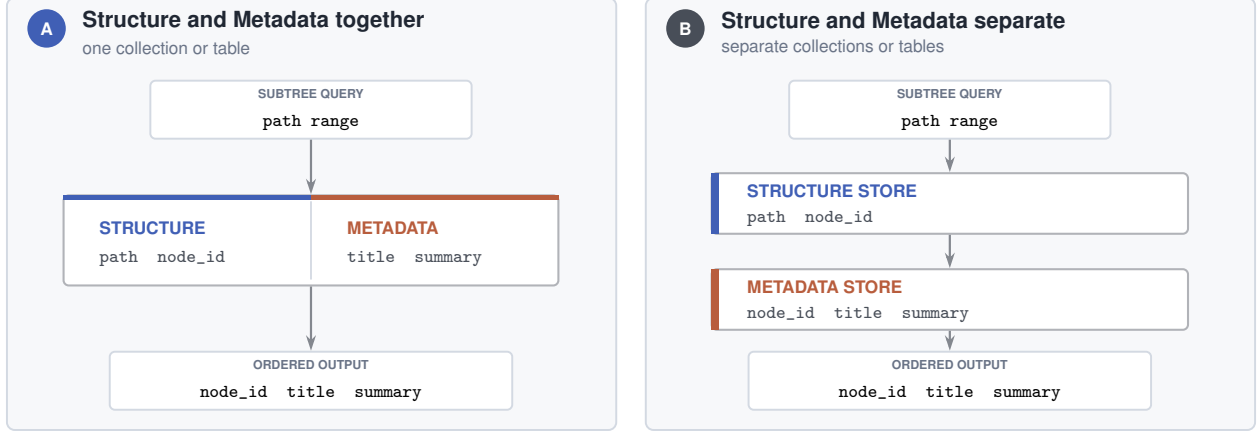


Figure 1: The two layouts used for `get_subtree`. Left: Structure and Metadata are together and are read by one ordered range scan. Right: Structure and Metadata are separate, so the Structure scan returns node IDs, Metadata is fetched in batches, and the records are restored to Structure order. Both paths return the same ordered rows. The arrows show query steps, not stored pointers. Text is separate in both layouts and is not read by this operation.

The colocated layout needs one ordered range scan. The separated layout first scans Structure for node IDs, fetches Metadata in batches, and restores the original Structure order. Thus both layouts return the same logical rows, but the separated layout adds database requests and client reassembly.

3 Experimental methodology

3.1 Workload and environment

The benchmark uses a ten-million-node tree and the same deterministic 200 subtree paths in MongoDB, PostgreSQL, and SQLite. The mean result size is 36,456.7 records per path. Both layouts are loaded side by side in each engine. The text store is identical and is not read by this operation. The two queries return the same ordered (`node_id,title,summary`) list for every path.

All runs use one client on the same dual-socket Intel Xeon Gold 6418H Linux host with 48 physical cores, 96 hardware threads, and 1 TiB RAM. MongoDB 7.0.34 and PostgreSQL 16.14 run as localhost Docker services. SQLite 3.37.2 runs in process. Database and operating-system caches are not cleared. Reported values therefore include the observed cache state. All latency values are rounded to one decimal place in the paper.

3.2 Layout-comparison protocol

Layout timing includes full result consumption and driver decoding. For the separated layout it also includes every Metadata batch and ordered reassembly; each Metadata request contains at most 1,000 node IDs. Each layout first runs on the same three paths without timing. Every path is then timed once per layout, and the layout measured first alternates by path. Comparisons are made within each engine; SQLite’s in-process values are not used as server-to-server measurements.

3.3 Cross-engine and causal protocol

After the layout comparison selects colocation, MongoDB and PostgreSQL run one full untimed sweep followed by three timed sweeps. Paths, query arms, and engine order are interleaved; the path order

rotates between repetitions. Every pair returns the same row count, logical values, and order. Repetitions are first averaged within each path, after which all 200 path means receive equal weight. Every arm accesses only the colocated Structure-and-Metadata collection or table; neither engine accesses the Text store or its index.

The initial breakdown has two additive stages. *Database fetch* includes server execution, localhost transfer, driver decoding, and materialization of returned records. *Client processing* includes normalization and cleanup of decoded objects. Because the first stage is composite, we then introduce zero-fetch exact-output, ID-only, cursor-batch, and scalar-count interventions. The two zero-fetch arms use 40 row-count-stratified paths and are accepted only when MongoDB examines zero documents and PostgreSQL performs zero heap fetches. The scalar-count experiment uses the direct MongoDB `count` command and all 200 paths over five repetitions, reducing each result to one scalar. MongoDB retains its native `COUNT` over `COUNT_SCAN`, while PostgreSQL retains `Aggregate` over `Index Only Scan`. PostgreSQL sequential scans and parallel gathers are disabled only in this mechanism-isolation experiment so both engines retain a covering-index access path.

A second control builds one million identical ordinals as integer, eight-byte string, and materialized-path-like long-string keys. PostgreSQL strings use `COLLATE C`, matching the real table. Thirty range counts per key shape are repeated five times. Finally, same-version MongoDB 7.0.34 and PostgreSQL 16.14 profiling containers each hold one million long-string keys. A single backend repeatedly counts the same 500,000-key range while CPU sampling identifies the functions on each engine’s per-key scan path.

4 Results

4.1 RQ1: Should Structure and Metadata be colocated?

Table 1: Subtree latency on the ten-million-node tree (lower is better). Every cell uses the same 200 paths, and both layouts return the same ordered records.

Engine	Structure and Metadata together		Structure and Metadata separate	
	P50 (ms)	P95 (ms)	P50 (ms)	P95 (ms)
MongoDB	23.2	283.4	49.9	581.9
PostgreSQL	12.9	146.7	40.5	474.1
SQLite	8.6	92.9	22.3	272.8

Keeping Structure and Metadata together gives lower P50 and P95 latency in all three engines. This comparison holds the returned fields, paths, and output order constant within each engine. Only the placement of Structure and Metadata changes.

The result answers RQ1 consistently across all three engines: colocating the fields required by the subtree response avoids Metadata batches and client reassembly. The remainder of the paper therefore uses this layout.

4.2 RQ2: End-to-end localization

Table 2 first locates the end-to-end difference using the additive stages defined in Section 3. This is a localization step, not yet a causal explanation.

Table 2: Subtree-latency breakdown with Structure and Metadata together. Each of the 200 paths has equal weight across three repetitions. The Share columns give each stage as a percentage of that engine’s total. Δ is MongoDB minus PostgreSQL, and the Δ share gives each stage’s percentage of the 18.1 ms total difference. Values are calculated before rounding.

Pipeline stage	MongoDB		PostgreSQL		Δ	
	Mean (ms)	Share	Mean (ms)	Share	Mean (ms)	Δ share
Database fetch	67.7	87.0%	54.8	91.9%	+12.8	70.8%
Client-side record processing	10.1	13.0%	4.8	8.1%	+5.3	29.2%
Unattributed time	0.0	0.0%	0.0	0.0%	< 0.1	< 0.1%
Total	77.8	100.0%	59.7	100.0%	+18.1	100.0%

Database fetch dominates the measured time in both engines. It accounts for 87.0% of the MongoDB total and 91.9% of the PostgreSQL total. MongoDB’s mean total is 18.1 ms higher. Database fetch contributes 12.8 ms, or 70.8%, of this difference. Client-side record processing contributes 5.3 ms, or 29.2%. The fetch bucket is still composite; the next experiment separates its server, result, and cursor mechanisms.

The MongoDB minus PostgreSQL total difference is positive and stable in all three timed repetitions: 18.5, 18.3, and 17.6 ms. Across the 200 per-path means, MongoDB/PostgreSQL is $1.84\times$ at P50, $1.81\times$ at P95, and $1.76\times$ at P99.

4.3 RQ2: Causal explanation

We now intervene on the query while holding its bounds and order fixed. *Exact-output covered* returns (`node_id`, `title`, `summary`) from a covering index. *ID-only covered* returns only `node_id`. Server-side EXPLAIN ANALYZE measurements use 40 row-count-stratified paths. Both arms examine the expected number of index keys; MongoDB examines zero documents and PostgreSQL performs zero heap fetches.

Table 3: Server execution under matched zero-fetch interventions. Each of the 40 paths has equal weight and returns 56,602.7 rows on average.

Arm	MongoDB (ms)	PostgreSQL (ms)	MongoDB/PostgreSQL
Exact-output covered	28.0	15.6	$1.79\times$
ID-only covered	20.6	7.7	$2.66\times$

Table 3 shows that document or heap access is not required for the gap: MongoDB remains $1.79\times$ slower when both engines use zero-fetch covering plans and return the exact output. The title/summary payload does not explain the gap either: covered minus ID-only costs 7.4 ms in MongoDB and 7.9 ms in PostgreSQL.

Two narrower interventions locate the remaining mechanism. First, increasing MongoDB’s requested batch size removes one `getMore` on average but does not reduce latency in either the ID-only or exact-output arm. Second, both engines return only a scalar count over all 200 original paths. MongoDB’s direct `count` command runs `COUNT` over `COUNT_SCAN` with zero documents examined; PostgreSQL is held to an index-only count with zero heap fetches. MongoDB takes 7.4 ms and PostgreSQL 4.3 ms. Their fitted per-key slopes are 0.190 and $0.112\mu\text{s}$, respectively, and the five repetition ratios remain between $1.69\times$ and $1.81\times$. The difference therefore survives removal of document access, per-row output, per-row BSON payload, and extra cursor round trips. Both engines still perform native count aggregation; the experiment does not assume that PostgreSQL obtains the count for free.

MongoDB profiler records on representative 5k, 12k, and 100k-row ranges report zero storage bytes read and zero storage-read time. For the ID-only covered arm, mean server CPU time is 15.8 ms and wall time is 15.3 ms. The lean MongoDB index is 280 MB versus 877 MB for PostgreSQL, so the result is not explained by storage reads or a larger MongoDB index.

Table 4: Scalar-count per-key slopes under key-shape control. PostgreSQL string indexes use binary `COLLATE C`.

Indexed key	MongoDB ($\mu\text{s}/\text{key}$)	PostgreSQL ($\mu\text{s}/\text{key}$)	Ratio
Integer	0.183	0.068	$2.68\times$
Eight-byte string	0.184	0.088	$2.10\times$
Long path-like string	0.189	0.089	$2.13\times$

Table 4 rules out path-string length as the source of MongoDB’s overhead: its slope is nearly invariant across the three key shapes.

Table 5: Matched names for the same per-key operations in the scalar control. Blue rows mark MongoDB’s bottleneck; cursor advance is the largest individual hotspot.

Logical operation	MongoDB profile	PostgreSQL profile
B-tree cursor advance	<code>__wt_btcur_next</code>	<code>_bt_next</code>
Index-entry check/read	<code>__wt_txn_read</code>	<code>_bt_checkkeys</code> , visibility map
Index-entry decode	<code>get_keyv/valuev</code> , <code>KeyString/RecordId</code>	<code>index_deform_tuple</code>
Executor handoff	<code>CountScan</code>	<code>ExecScan</code>
Count accumulation	<code>CountStage</code>	<code>int8inc</code>

PostgreSQL therefore performs the same logical operations. Across these matched steps, MongoDB’s excess CPU is distributed primarily across cursor advance, entry extraction/decoding, and classic executor bookkeeping. This locates the base index-processing bottleneck; client record processing adds the separate 5.3 ms difference in Table 2.

5 Conclusion

The two research questions have separate answers. For physical design, Structure and Metadata should be colocated for this descendant-range workload: doing so lowers P50 and P95 latency in MongoDB, PostgreSQL, and SQLite. For the cross-engine comparison, document fetch, returned metadata, and an extra cursor round trip cannot explain MongoDB’s higher latency: the gap remains when both engines scan the same logical index range and return only a count. The direct-count path is $1.71\times$ slower on the same 200 paths, and key-shape controls show that long path strings are not the cause. PostgreSQL also advances a B-tree cursor and runs an executor and aggregate. MongoDB’s bottleneck is the heavier per-key WiredTiger cursor/key-extraction and classic `COUNT_SCAN` bookkeeping path in the scalar control. Client record processing adds a further 5.3 ms to the end-to-end difference.